

Caminhos Mínimos em Grafo Viário Real da RMSP: Comparação entre BSP e Fan-out de Origens em Contêiner Docker

Luiz Soares

Programa de Pós-Graduação em Ciência da Computação – UFLA

Junho de 2026

Resumo

Este artigo apresenta um estudo experimental de processamento de grafos em contexto de Big Data, tendo como caso de uso o cálculo de caminhos mínimos no grafo viário real da Região Metropolitana de São Paulo (RMSP). O problema é derivado de uma pesquisa sobre localização de estações de recarga para veículos elétricos, na qual a etapa de distâncias rodoviárias é um dos principais gargalos computacionais. O grafo de entrada é o GraphML real da RMSP, com 1.150.979.779 bytes, 770.053 vértices e 2.252.485 arestas dirigidas. Comparam-se duas estratégias: uma formulação vertex-centric inspirada no modelo Pregel/BSP e uma estratégia fan-out, na qual as origens são processadas independentemente por Dijkstra. A validação obteve erro absoluto máximo igual a zero entre as matrizes produzidas.

1 Introdução

Processamento de grafos é uma classe recorrente em Big Data porque combina grande volume, baixa localidade de acesso e dependência iterativa entre vértices. Em malhas viárias metropolitanas, tarefas aparentemente simples, como calcular distâncias entre pontos relevantes, passam a envolver milhões de arestas e múltiplas execuções de caminhos mínimos.

O problema estudado neste trabalho é o cálculo de Single-Source Shortest Path (SSSP) repetido em um grafo rodoviário dirigido e ponderado da RMSP. Essa matriz de distâncias é usada como insumo para modelos de localização de infraestrutura de recarga: zonas de origem, candidatos e concorrentes precisam ser comparados por custo de deslocamento realista, e não apenas por distância euclidiana.

Esse contexto é especialmente relevante para a RMSP. A literatura recente sobre estações de recarga enfatiza que motoristas escolhem onde carregar considerando distância, preço, disponibilidade, tempo de espera e conveniência [8, 10, 9]. Logo, a qualidade da matriz de distâncias afeta diretamente o nível inferior desses modelos: se o custo de deslocamento entre origem, estação e destino for mal estimado, a alocação de fluxo e a escolha das localizações tendem a ser distorcidas. A Pesquisa Origem-Destino 2023 do Metro de São Paulo fornece a base de fluxos e zonas de tráfego, enquanto o OpenStreetMap fornece a malha viária usada para transformar pontos geográficos em custos de percurso [11, 12, 13].

As contribuições do trabalho são:

- preparação do GraphML real da RMSP para o contrato Parquet usado pelos workloads;
- implementação de dois workloads comparáveis para SSSP;
- validação cruzada das matrizes de distância;
- análise de tempo, throughput, speedup e eficiência.

2 Fundamentação

O modelo MapReduce popularizou o processamento distribuído em larga escala para tarefas com etapas de mapeamento e agregação [1]. O Spark introduziu RDDs e processamento em memória, diminuindo o custo de workloads iterativos em relação a pipelines puramente baseados em disco [2].

Para grafos, Pregel consolidou o paradigma “think like a vertex”: cada vértice mantém estado local, envia mensagens a vizinhos e sincroniza em supersteps [3]. GraphX levou essa ideia para o ecossistema Spark, representando grafos sobre RDDs e oferecendo operações específicas para processamento distribuído de grafos [5].

O algoritmo de Dijkstra resolve SSSP em grafos com pesos não negativos [4]. Quando há muitas origens, uma alternativa é paralelizar as origens: cada tarefa executa Dijkstra sobre a mesma estrutura de grafo. Essa estratégia é eficiente quando o grafo cabe em memória, mas pode pressionar broadcast e memória em execuções maiores.

3 Dados

A entrada única do experimento é o grafo viário real da RMSP em GraphML, produzido no pipeline da dissertação a partir do OpenStreetMap e processado com OSMnx [13]. O arquivo `data/rmsp_graph/road_network.graphml` possui 1.150.979.779 bytes.

O script `bin/prepare-rmsp-graph.py` percorre o GraphML por streaming e gera duas tabelas Parquet:

- `vertices(id, osm_id, lat, lon)`: 770.053 vértices;
- `edges(src, dst, length_m, travel_time_s)`: 2.252.485 arestas dirigidas.

Tabela 1: Entrada real do experimento.

Artefato	Valor
GraphML	1.150.979.779 bytes
Vértices	770.053
Arestas dirigidas	2.252.485
Parquet de vértices	20 MB
Parquet de arestas	47 MB

Como o GraphML contém arestas paralelas entre alguns pares de vértices, os workloads consolidam essas paralelas pelo menor comprimento antes de montar a estrutura de caminhos mínimos. Essa operação preserva a distância mínima entre pares de nós.

4 Arquitetura Experimental

A arquitetura segue o requisito da disciplina: repositório autocontido, execução exclusivamente via Docker Compose (contêiner único), scripts de execução e resultados reproduzíveis. O paralelismo é obtido distribuindo as origens entre processos trabalhadores; o grafo é compartilhado por `fork` (copy-on-write), evitando reserializar milhões de arestas. O fluxo é:

1. converter o GraphML real para Parquet;
2. carregar vértices e arestas;
3. executar A-BSP e B-FANOUT para as mesmas origens;
4. registrar `benchmark/metrics.csv`;
5. validar `distances_a.npz` e `distances_b.npz`;
6. gerar gráficos, relatório, artigo e slides.

5 Workloads

5.1 A-BSP: Vertex-centric

O Workload-A representa o grafo como conjunto de vértices e arestas. Para cada origem, a distância inicial é zero no vértice fonte e infinita nos demais. Em cada rodada conceitual, vértices alcançáveis relaxam arestas, mensagens são agregadas por destino usando mínimo, e o estado do vértice é atualizado se uma distância menor for encontrada.

5.2 B-FANOUT: Origens independentes

O Workload-B usa a estratégia oposta: o grafo é convertido para CSR e cada origem executa Dijkstra de forma independente. Não há mensagens entre tarefas nem barreiras iterativas globais. O paralelismo natural está nas linhas da matriz de distâncias.

6 Métricas

As métricas coletadas por repetição são tempo de parede, throughput em origens por segundo, throughput aproximado de relaxações por segundo, número de supersteps conceituais, speedup e eficiência paralela. Cada configuração foi executada três vezes.

7 Resultados

Os experimentos foram executados no grafo real da RMSP, com 64 origens e 3 repetições para cada configuração, variando 1, 2 e 4 processos trabalhadores. A validação produziu matrizes 64×770053 e erro absoluto máximo igual a zero entre A-BSP e B-FANOUT.

Tabela 2: Resultados no grafo viário real da RMSP (64 origens).

Workload	W	Tempo (s)	Speedup	Orig./s
A-BSP	1	43.087 ± 0.309	1.000	1.49
A-BSP	2	24.209 ± 0.339	1.780	2.64
A-BSP	4	14.099 ± 0.378	3.056	4.54
B-FAN	1	5.588 ± 0.035	1.000	11.45
B-FAN	2	3.346 ± 0.025	1.670	19.13
B-FAN	4	1.901 ± 0.017	2.939	33.67

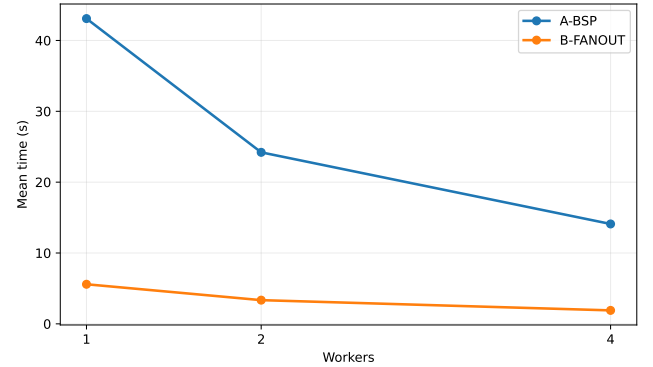


Figura 1: Tempo de execução no grafo real da RMSP.

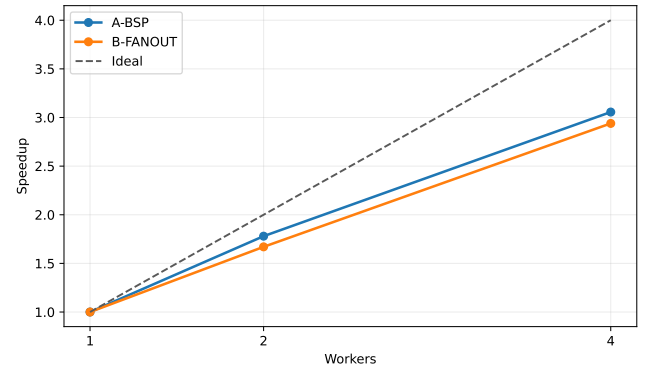


Figura 2: Speedup por número de workers no grafo real.

8 Análise

O Workload-B foi cerca de 7,4 vezes mais rápido que o Workload-A em termos absolutos. Esse resultado é coerente com o desenho do experimento: quando o grafo cabe em memória, executar Dijkstra compilado por origem evita as barreiras de coordenação do modelo BSP. Ambos os workloads apresentaram speedup próximo de $3\times$ com 4 processos (eficiência de 0,76 para A e 0,73 para B), com desvios padrão inferiores a 3% da média. A eficiência cai suavemente de 2 para 4 processos por causa da contenção de largura de banda de memória e do custo de coletar as linhas completas da matriz de distâncias no processo pai. Como as origens são independentes, o workload é essencialmente paralelizável quando o paralelismo é obtido por processos, e não por threads limitadas pelo GIL do Python.

9 Conclusão

O trabalho compara duas estratégias de SSSP no grafo viário real da RMSP. A validação mostrou equivalência numérica entre as matrizes produzidas e os resultados indicam vantagem do fan-out para as origens executadas. A escolha entre BSP e fan-out depende do regime de dados: quando memória e broadcast são viáveis, fan-out tende a ser simples e eficiente; quando a replicação do grafo se torna limitante, o particionamento BSP passa a ser uma alternativa relevante.

Referências

- [1] Dean, J. and Ghemawat, S. MapReduce: Simplified Data Processing on Large Clusters. *Communications of the ACM*, 2008.
- [2] Zaharia, M. et al. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. NSDI, 2012.
- [3] Malewicz, G. et al. Pregel: A System for Large-Scale Graph Processing. SIGMOD, 2010.
- [4] Dijkstra, E. W. A Note on Two Problems in Connection with Graphs. *Numerische Mathematik*, 1959.
- [5] Gonzalez, J. E. et al. GraphX: Graph Processing in a Distributed Dataflow Framework. OSDI, 2014.
- [6] Anjos, M. F. et al. Integrated Location, Sizing, and Pricing for EV Charging Stations. In: *Combinatorial Optimization and Applications*. Springer, 2024.
- [7] Adil, M. et al. Optimal Location and Pricing of Electric Vehicle Charging Stations Using Machine Learning and Stackelberg Game. *IEEE Transactions on Industry Applications*, 2024.
- [8] Makhlouf, W.; Kchaou-Boujelben, M.; Gicquel, C. A bi-level programming approach to locate capacitated electric vehicle charging stations. CoDIT, 2019.
- [9] Zhao, Y. et al. Deployment of the Electric Vehicle Charging Station Considering Existing Competitors. *IEEE Transactions on Smart Grid*, 2020.
- [10] Xiong, Y. et al. Optimal Electric Vehicle Fast Charging Station Placement Based on Game Theoretical Framework. *IEEE Transactions on Intelligent Transportation Systems*, 2018.
- [11] Metro de São Paulo. *Pesquisa Origem e Destino 2023: Síntese das Informações*. Companhia do Metropolitano de São Paulo, 2024.
- [12] OpenStreetMap contributors. *OpenStreetMap*. Dados sob licença ODbL, 2025.
- [13] Boeing, G. OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks. *Computers, Environment and Urban Systems*, 2017.